

Infografía

Módulo 3.1 — Rasterización (parte 2)

Círculos, triángulos, anti-aliasing

UPB · Gestión II/2025

Dónde estamos

Ya rasterizamos **líneas** de tres maneras:

- **DDA** — el algoritmo "obvio", con floats y redondeo.
- **Bresenham (un octante)** — solo enteros, un caso.
- **Bresenham (8 octantes)** — el caso general.

Hoy: subimos de dimensión

Líneas (1D) → Círculos (curva) → Triángulos rellenos (2D) → Anti-aliasing (intensidad)

El hilo conductor

Todos los algoritmos de hoy comparten **la misma idea base** que Bresenham:

```
una variable de decisión incremental  
+ decide qué pixel encender a continuación  
+ usa solo enteros (o casi)
```

Cambia el objeto que rasterizamos (curva, región, línea suave) — no la idea.

El GPU hace esto por debajo, en hardware, miles de millones de veces por segundo.

1. Midpoint Circle

`ex4_circle.py`

El problema

Queremos prender los píxeles que aproximan la curva

$$x^2 + y^2 = r^2$$

Opción ingenua: `for x in -r..r: y = sqrt(r2 - x2)`.

Problemas:

- `sqrt` por cada paso → costoso.
- Distribución no uniforme de píxeles cerca de los polos.

Mejor idea: la misma de Bresenham — variable de decisión entera.

El truco de la simetría

Un círculo tiene **8 ejes de simetría**.

Si conocemos los píxeles de **un solo octante** (de 0° a 45°), los otros 7 salen reflejando.

```
def plot_8_symmetric(xc, yc, x, y):  
    return [  
        (xc+x, yc+y), (xc-x, yc+y),  
        (xc+x, yc-y), (xc-x, yc-y),  
        (xc+y, yc+x), (xc-y, yc+x),  
        (xc+y, yc-x), (xc-y, yc-x),  
    ]
```

A diagram showing the 8 axes of symmetry for a circle. The axes are labeled with cardinal and ordinal directions: N (North), NNE (North-Northeast), E (East), SSE (South-Southeast), S (South), SSW (South-Southwest), W (West), and NNW (North-Northwest). The labels are arranged in a circular pattern around a central point.

1/8 de cómputo, 8/8 de círculo

La variable de decisión

Arrancamos en el "norte" del círculo: $(x, y) = (0, r)$.

En cada paso: avanzamos x en 1. Y y :

- **se queda igual** si el píxel (x, y) sigue del lado de adentro
- **baja en 1** si el píxel $(x, y-1)$ está más cerca de la curva

```
p = 1 - r                # decisión inicial
while x < y:
    x += 1
    if p < 0:              # adentro: solo avanzar x
        p += 2*x + 1
    else:                  # afuera: bajar y también
        y -= 1
        p += 2*(x - y) + 1
```

*Solo sumas y restas enteras. **Cero floats, cero sqrt.***

Demo en vivo

```
cd 3._arcade/3.1_algorithms  
uv run ex4_circle.py
```

Cosas para mirar en pantalla:

- Los 8 octantes se forman simultáneamente (un paso del bucle → 8 píxeles).
- Donde `x == y` (la diagonal), los octantes se solapan — algunos píxeles aparecen 2 veces.
- El círculo es **discreto**: hay un escalón visible. Lo arreglamos en Wu (al final).

Extensiones

Si sobra tiempo (o como ejercicio):

1. **Concéntricos** — dibujar 5 círculos con distintos r y mismo centro.
2. **Fuerza bruta vs. midpoint** — implementar $y = \sqrt{r^2 - x^2}$ y comparar.
3. **Círculo relleno** — en cada paso, en vez de 8 píxeles dibujar las **4 líneas horizontales** que conectan los pares simétricos.
4. **Ellipse** — midpoint ellipse (dos regiones, pendiente $|m| = 1$ como frontera).

2. Triangle Fill

`ex5_triangle_fill.py`

¿Por qué triángulos?

Hasta acá rasterizamos **curvas 1D** (líneas, círculos).

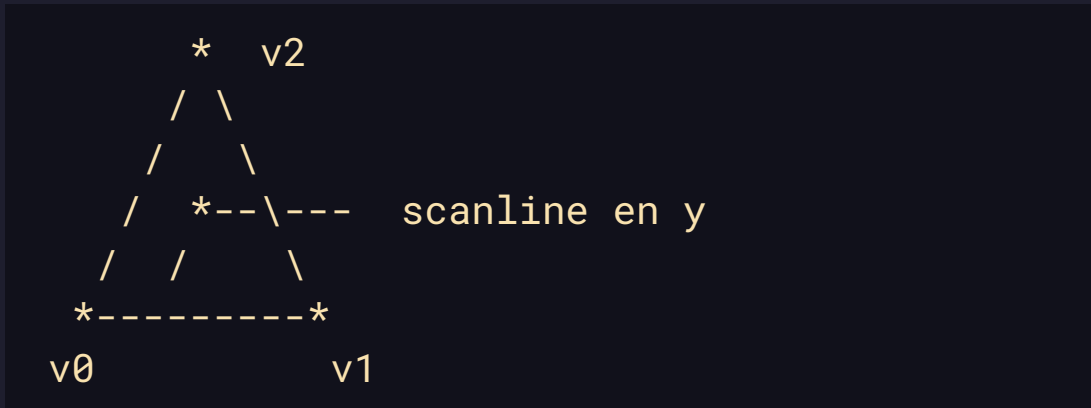
El triángulo es **la primitiva 2D fundamental** de los gráficos por computadora:

- Cualquier malla 3D es una colección de triángulos.
- El **rasterizer del GPU** convierte cada triángulo en píxeles.
- Texturas, iluminación, shaders: todo opera **por píxel dentro del triángulo**.

Si entienden este algoritmo, entienden cómo el GPU pinta un juego 3D.

La técnica: scanline fill

Idea: para cada **fila horizontal** (scanline) que cruza el triángulo, encontrar las **dos aristas activas** y pintar la franja entre ellas.



Tres pasos:

1. **Ordenar** los vértices por y ascendente.
2. **Partir** el triángulo en mitad inferior y superior (en $y == y_1$).
3. Para cada y : calcular x_a , x_b por interpolación lineal, llenar $[x_a..x_b]$.

Interpolación de bordes

Para encontrar dónde una arista $(x_a, y_a) \rightarrow (x_b, y_b)$ cruza la altura y :

```
def edge_x(ya, yb, xa, xb, y):  
    if yb == ya:  
        return float(xa)  
    t = (y - ya) / (yb - ya)    # proporción [0..1] a lo largo de la arista  
    return xa + (xb - xa) * t    # interpolación lineal
```

Esto es **interpolación lineal** — el mismo concepto que aparecerá cuando hagamos shading (Gouraud) y mapeo de texturas. Acá interpolamos solo posición; en el GPU también color y UVs.

El bucle completo

```
v0, v1, v2 = sorted([p0, p1, p2], key=lambda p: p[1])

# mitad inferior: y0 <= y < y1
for y in range(y0, y1):
    xa = edge_x(y0, y2, x0, x2, y)    # arista larga v0->v2
    xb = edge_x(y0, y1, x0, x1, y)    # arista corta v0->v1
    for x in range(int(min(xa, xb)), int(max(xa, xb)) + 1):
        pixels.append((x, y))

# mitad superior: y1 <= y <= y2
for y in range(y1, y2 + 1):
    xa = edge_x(y0, y2, x0, x2, y)
    xb = edge_x(y1, y2, x1, x2, y)
    ...
```

Casos degenerados

Estos no son "edge cases académicos". Son **el 95% de los bugs** de un rasterizador real:

- Triángulo **casi-horizontal** (`y0 == y1`) → la mitad inferior tiene 0 scanlines, hay que cuidarse de dividir por 0 en `edge_x`.
- Dos vértices **repetidos** → triángulo degenerado a una línea.
- Vértices con **misma y** después de ordenar → ¿qué arista es la "larga"?
- Triángulo con un vértice **fuera de pantalla** → clipping (no lo hacemos hoy).

El día que escriban un shader y vean "T-junctions" o aristas con gap, vuelvan a esta diapositiva.

Demo

```
uv run ex5_triangle_fill.py
```

Para mirar:

- Bordes "dentados" — el triángulo es **discreto**, igual que el círculo.
- El reparto entre mitad inferior / mitad superior es **invisible** en pantalla — bien, así debe ser.
- Cantidad de píxeles $\approx$ área del triángulo. Para escena 3D real: **miles de millones por frame**.

3. Wu – Anti-Aliasing

`ex6_wu_antialiasing.py`

El problema visual

Todo lo que rasterizamos hasta acá tiene **bordes escalonados** (*aliasing*, "jaggies").



Pasa porque cada pixel está **prendido o apagado**, sin medias tintas. La línea matemática no encaja exactamente en la grilla → el ojo ve la escalera.

Anti-aliasing: prender pixels con intensidad fraccional

La idea de Wu

Bresenham/DDA: **un** pixel por paso, intensidad 1.

Wu: **dos** pixels por paso (los dos vecinos verticales), con intensidades que **suman 1**:

```
⋮ ← y_int + 1    (intensidad = frac)
⋮ ← y_int        (intensidad = 1 - frac)
```

Donde **frac** es la parte fraccional de la **y** ideal en esa columna.

*El ojo **integra** los dos píxeles en un gris intermedio. El escalón desaparece visualmente.*

Conceptualmente: convolución

Lo que estamos haciendo es **convolución implícita** de la línea ideal con el kernel del pixel:

- "¿Qué fracción del píxel cubre la línea?" → esa es su intensidad.
- En realidad usamos una aproximación (vecino más cercano), no el área exacta — barato y suficiente.

Esto es **la idea central de todo el anti-aliasing moderno**: MSAA, FXAA, TAA, SDF fonts — todos están dando intensidades fraccionales en la frontera.

El código

```
gradient = dy / dx
intery = y0 + gradient
for x in range(x0 + 1, x1):
    y_int = int(intery)
    frac = intery - y_int
    plot(x, y_int,      1 - frac)    # pixel de abajo
    plot(x, y_int + 1, frac)        # pixel de arriba
    intery += gradient
```

Más boilerplate para manejar el caso `|m| > 1` (intercambiar ejes) — ya está hecho en el archivo.

Demo lado a lado

Idealmente: dos terminales abiertas.

```
uv run ex3_bresenham_full.py    # escalonado puro
uv run ex6_wu_antialiasing.py   # con anti-aliasing
```

Las dos líneas son **las mismas matemáticamente**. La diferencia visual es la mejor demostración del módulo.

El "costo" de Wu: ~2x los píxeles dibujados y un poco de aritmética flotante. Aceptable para CPU; en GPU se hace gratis vía MSAA en hardware dedicado.

Resumen del módulo

Algoritmo	Primitiva	Truco clave
DDA	línea	floats + redondeo
Bresenham	línea	variable de decisión entera
Midpoint	círculo	decisión + simetría 8×
Scanline	triángulo	interpolación lineal por fila
Wu	línea AA	2 px/paso con intensidad fraccional

Los 3 algoritmos de hoy son la base de cómo el GPU pinta una escena 3D.

¿Preguntas?

Después del break — Game loop, eventos, sprites

```
cd 3._arcade/3.1_algorithms && uv run ex4_circle.py
```